Kommunikation in Verteilten Systemen
Wintersemester 2026/27
Übungszettel 01

Prof. Dr. Peter Martini, Dr. Matthias Frank, Lennart Buhl, Dylan Stogianni
Ellen Bundschuh (Mo), Maria Alexander (Di), Aura Pop (Mi)

- Herausgegeben am: **2026-10-23**
- Einreichen der Lösungen:
  - Bitte **spätestens bis 2026-11-01 23:59 (bis Sonntag, normal wird in Zukunft voraussichtlich bis Freitag sein).**
  - Die Einreichung erfolgt über das entsprechende GitLab-Projekt auf den dem Zettel zugehörigen Branch. **Bitte legen sie die Lösungen direkt auf den Branch und erstellen sie keine unnötigen Unterordner.** Einreichungen über andere Medien (bspw. E-Mail) werden nicht akzeptiert. Es zählt die zur Deadline vorliegende Version.
  - **Alle Skripte und Quelltext-Dateien**, die zur Lösung praktischer Programmieraufgaben nötig sind müssen vorliegen. Der Aufruf des Codes muss dabei mittels einem Aufruf von `run.sh` möglich sein.
  - Theoretische Aufgaben eines Zettel müssen in **einer PDF-Datei** abgelegt werden und alle Dokumentationen und Erklärungen, sowie **allen relevanten Grafiken und Plots** enthalten.
  - Als Includes/Imports für ihre praktischen Programme sind nur solche Bibliotheken erlaubt, welche den Kernaspekt der Aufgabe nicht vorwegnehmen.
- Präsentation der Lösungen: **In den Tutorien 2026-11-02 bis -04**

Aufgabe 1: Flavius

Übungszettel müssen in Gruppen von zwei oder drei Studis gelöst werden. Einzelabgaben sind aus Gründen des Teamworks und Workload des Tutorinnenteams nicht vorgesehen und nicht gestattet.

Die Registrierung der Kleingruppen zur Abgabe hierzu läuft über **Flavius** (nur aus dem Informatik-Netz oder per VPN zu erreichen; manchen vielleicht bereits aus Erstsemestervorlesungen bekannt): https://flavius.cs.uni-bonn.de/

Bei den Kleingruppen zur Übungsabgabe ist es ratsam, dass alle Teilnehmer den gleichen Übungstermin haben. Die Anmeldungen sollten so bald wie möglich erfolgen.

Zur Registrierung in Flavius ist ein **funktionierender Informatik-Account** notwendig, denn der Login-Prozess in Flavius läuft über das GitLab der Informatik. Bei Problemen mit eurem Informatik-Account wendet euch bitte an die Systemgruppe: https://gsg.cs.uni-bonn.de. Bei anderweitigen Problemen wendet euch an orga-kivs@lists.iai.uni-bonn.de; dies ist dem Kontaktieren von Einzelpersonen des Kivs-Teams vorzuziehen! Wenn Einzelpersonen des Kivs-Teams direkt kontaktiert werden kann es vorkommen dass eure Anfragen (deutlich) verzögert oder im schlimmsten Fall gar nicht beantwortet werden können.

Der Workflow ist folgender: Flavius-Website aufrufen, über den GitLab-Account einloggen, auf das entsprechende Modul klicken und dann unten bei "Join Tutorial via Token"den per E-Mail erhaltenen Token eingeben. Anschließend besteht entweder die Möglichkeit eine Kleingruppe zu erstellen oder via Abgabegruppen-Join-Token (den ihr von Kommilitonen welche bereits eine Abgabegruppen erstellt haben bekommt) einer anderen Kleingruppe beizutreten. Am Ende sollte dann Zugriff auf das Repo der entsprechenden Abgabegruppe im GitLab (`https://gitlab.cs.uni-bonn.de`) bestehen.

Pro Übungszettel gibt es einen Merge Request auf einem eigenen Branch, in welchen die Lösungen commited werden müssen.

Aufgabe 2: Hello World [5 Punkte]

Sie werden für praktische Aufgaben jeweils ein Programm schreiben, welches seine konkrete Aufgabe anhand des ersten Kommandozeilenparameters erkennt. Für Python und Java finden sich hier Beispielimplementierungen dieses Verhaltens: `https://gitlab.com/tallion/kivs-sample-projects`. Wir schränken die Auswahl an Programmiersprachen zwar nicht explizit ein, es ist jedoch dafür Sorge zu tragen, dass eine Nachvollziehbarkeit durch unsere Tutorinnen gewährleistet ist! Alle Programme müssen unter Linux lauffähig sein.

Zur Evaluation der Programmieraufgaben werden automatisierte Tests eingesetzt, deswegen ist es wichtig dass die dargelegte Struktur eingehalten wird. Als Angebot für die Bearbeitung werden diese Tests jeweils in die PDF-Datei des Aufgabenzettel eingebettet[1] zur Verfügung gestellt. Die entsprechende `test.py` können sie jeweils in ihr Repository übernehmen, es handelt sich dabei um ein freiwilliges Angebot um die Lösung vorab auf Konsistenz mit dem Schema und teils bereits auf ihre Funktionalität zu prüfen.

Es gibt einige Kriterien, die die Struktur erfüllen muss, damit die Abgabe getestet werden kann:

- Es muss eine ausführbare Datei `run.sh` geben, welche die Lösung aufruft.
- Wenn Sie ihr Programm kompilieren müssen, so sollte es ein `Makefile` geben das die entsprechenden Befehle ausführt.
- Ihr Programm kann beliebige Ausgaben machen. Zeilen die Lösungen enthalten müssen jedoch auf `stdout` erfolgen und mit der Zeichenkette `SOLUTION:` anfangen.

Diese Aufgabe dient dazu, dass Sie sich mit dem Testframework vertraut machen sollen. (Bitte beachten, dass alle Zeilen ohne das Prefix `SOLUTION:` ignoriert werden.)

Schreibe ein Programm, das einen Befehl (square oder cube) und eine Zahl x als Eingabe einnimt und dafür entweder x^2 oder x^3 berechnet.

Sie können das Test-Framework auch lokal nutzen indem sie es mittels `pip install stef` installieren und (nach ggf. `make`) mittels `stef_runtests . .` aufrufen, wobei mittels eines optionalen Parameters `--testgroups TESTGROUP` auch die jeweilige Testgroup explizit angegeben werden kann.

Eingabedefinition:

- Erster Parameter (a_1): `square` $\vee$ `cube`
- Zweiter Parameter (a_2): Zahl x, die als Eingabe dienen soll

Ausgabedefinition: Resultat der Rechnung

[1] in Browsern wird das für gewöhnlich nicht angezeigt, man muss die Datei dazu runterladen und mit einem PDF-Programm öffnen, Evince/Atril, Okular und Acrobat Reader funktionieren auf jeden Fall

Aufgabe 3: Wireshark Analyse [5 Punkte]

Die Fähigkeit, die über ein Kommunikationsnetzwerk gesendeten Datenpakete zu analysieren, ist eine wesentliche Fähigkeit für die Analyse bestehender Netzwerkprotokolle und Anwendungen. Angenommen, der Netzwerkverkehr ist unverschlüsselt, können mit einem Netzwerksniffing-Tool wie Wireshark umfassende Studien durchgeführt werden. Wireshark (`https://www.wireshark.org`) ist ein leistungsstarkes Tool, das in der Lage ist, den Netzwerkverkehr auf verschiedene Weise zu erfassen und darzustellen. In dieser Aufgabe sollen Sie sich mit diesem Tool vertraut machen.

1. Laden Sie Wireshark von der angegebenen Quelle herunter und machen Sie sich mit diesem Tool vertraut. Erfassen Sie einige verschiedene Arten von Netzwerkverkehr (HTTP, IMAP / POP3 und Secure Shell (ssh)-Anmeldung). Was beobachten Sie?
2. Eingebettet in diese PDF-Datei finden Sie eine Datei, die mit Wireshark erfasst wurde. Erklären Sie, was Sie in der Datei sehen können. Einige wichtige Fragen sind:
   - Was genau passiert in den 45 Schritten, die im Protokoll angezeigt werden?
   - Welche Art von Verbindung wurde hergestellt?
   - Welche Protokolle welcher OSI-Schicht können beobachtet werden?
3. Welche Arten von Geräten (Hersteller) haben auf Schicht 2 miteinander kommuniziert? Wie kann Wireshark diese identifizieren?
4. Ist es möglich herauszufinden, welches Betriebssystem auf dem Host mit der IP 131.220.6.214 läuft?

Aufgabe 4: Analyse von Pfaden im Internet [5 Punkte]

Mit `traceroute`/`tracepath` (Linux) oder `tracert` (Windows) können Details über den Pfad von einer Quelle zu einem Ziel ermittelt werden. Probieren Sie das Tool mit einigen Adressen aus. Verfolgen Sie den Pfad zu unserem Webserver (`net.cs.uni-bonn.de`) von Ihrem Computer zu Hause und von einer Verbindung im Institut aus.

1. Was beobachten Sie?
2. Wie funktioniert `trace[rt|route|path]`?
   - Was ist ein ICMP-Paket?
   - Wer sendet diese Pakete?

Aufgabe 5: Grenzen der Kommunikation [5 Punkte]

Betrachten Sie ein Netzwerk mit drei Knoten A, B und C (siehe Abbildung). Die Knoten A und B sowie B und C können Daten austauschen. Alle Knoten arbeiten halbduplex, das bedeutet, ein Knoten kann Daten senden und empfangen, jedoch nur in eine Richtung gleichzeitig (nicht gleichzeitig senden und empfangen). Wenn A Daten an C senden möchte, sendet es sie an B. B leitet die Daten dann an C weiter. Dasselbe passiert, wenn C Daten an A senden möchte.



1. Wie viele Nachrichten sind erforderlich, um ein Paket von A nach C und ein weiteres von C nach A zu senden?
2. Machen Sie sich mit dem Konzept des „Network Coding" vertraut. Können Sie die Nachrichtenanzahl mit Network Coding reduzieren? (Falls ja, wie? Falls nein, warum nicht?)